

E-Commerce Dark Pattern Detection Using NLP and Machine Learning

Problem Statement:

E-commerce websites contain a large amount of text such as product descriptions, offers, notifications, pop-up messages, stock information, and promotional messages. Some of these messages may use persuasive techniques to influence users to make quick purchasing decisions.

For example, an online shopping website may display messages such as:

- "Hurry! Only 2 items left!"
- "Limited time offer!"

Such messages can create a feeling of urgency, scarcity, or social pressure. These types of potentially manipulative interface and content techniques are commonly referred to as dark patterns.

Two machine learning algorithms, **Logistic Regression** and **Naive Bayes**, are implemented and compared. The better-performing model is then used as the final dark-pattern detector.

It is important to note that the system identifies text that **resembles potentially manipulative dark-pattern language** based on patterns learned from the dataset. It does not verify whether a statement is actually true or false. For example, if a website says "*Only 2 items left*", the model can identify the language as similar to scarcity-based dark-pattern text, but it cannot determine whether the website actually has two items remaining.

Objectives:

The main objective of this project is to develop a simple NLP-based system for detecting dark-pattern (manipulative) language in e-commerce text.

The specific objectives are:

1. To study the dark-pattern problem – Understand how e-commerce messages use tactics like urgency, scarcity, and social proof to influence users.

2. To prepare and analyze the dataset – Examine its structure, class distribution, and any missing or duplicate records.
3. To preprocess the text – Clean the text (lowercase, remove special characters/extra spaces) to prepare it for modeling.
4. To extract features using TF-IDF – Convert the cleaned text into numerical vectors suitable for machine learning.
5. To train classification models – Build Logistic Regression and Multinomial Naive Bayes models.
6. To evaluate and compare models – Use accuracy, precision, recall, F1-score, and confusion matrix to assess performance.
7. To select the best-performing model for the final detector.
8. To build a simple detector function that classifies new e-commerce text as *Dark Pattern* or *Not Dark Pattern*.
9. To analyze limitations – Review misclassifications and note possible improvements.

Dataset:

The dataset used in this project is the E-Commerce Dark Pattern Text Dataset, stored as dataset.csv. It consists of e-commerce text messages labeled to indicate whether they contain dark-pattern (manipulative) language.

Source:

[Dark Patterns](#)

Size:

The dataset contains 2,356 records and 4 original columns: page_id, text, label, and Pattern Category. No missing values, duplicate rows, or duplicate text entries were found. A clean_text column was added during preprocessing. The dataset also contains 1,248 unique page IDs, with some pages contributing multiple examples.

Attributes:

Category	Description
Page id	Identifies the web page the text example came from; not used as a model feature
text	The main input — the actual e-commerce message analyzed
label	Target variable for binary classification: 0 = Not Dark Pattern, 1 = Dark Pattern
Pattern category	Specifies the type of manipulation technique used, where applicable

Classes/Categories:

The dataset is perfectly balanced for the main target variable:

Label	Meaning	Records
0	Not Dark Pattern	1,178 (50%)
1	Dark Pattern	1,178 (50%)

The Pattern Category column breaks the dark-pattern class into finer categories: Scarcity (418), Social Proof (312), Urgency (210), Misdirection (195), Obstruction (27), Sneaking (12), and Forced Action (4). Because several of these categories have very few examples, the project uses **binary classification** (dark pattern vs. not) rather than predicting all eight categories individually.

Data Preparation:

Basic text preprocessing was applied to create a clean text column:

1. Lowercasing — e.g., "HURRY! Only 2 Items Left!" → "hurry only 2 items left"
2. Removing special characters — e.g., punctuation like !, ?, ., \$ stripped
3. Removing extra spaces — multiple spaces collapsed into one
4. Preserving stopwords — words like "only" and "left" were kept, since they can carry scarcity-related signal

Methodology:

The overall methodology followed in this project is:

Dataset → Text Preprocessing → Train-Test Split → TF-IDF → Machine Learning Model → Prediction → Evaluation

- **Text Preprocessing:**

Raw e-commerce text may contain uppercase letters, punctuation, unnecessary spaces, and other characters.

The cleaning process:

1. Converts text to lowercase.
2. Removes characters other than letters, numbers, and spaces.
3. Removes extra spaces.
4. Returns the cleaned text.

- **Feature Extraction Using TF-IDF:**

Machine learning algorithms cannot directly understand sentences.

Therefore, the text needs to be converted into numbers.

TF-IDF converts text into numerical values so that a machine learning model can learn patterns from the text.

The training data was fitted and transformed using:

```
X_train_tfidf = tfidf.fit_transform(X_train)
```

The test data was transformed using:

```
X_test_tfidf = tfidf.transform(X_test)
```

- **Machine Learning Models:**

1. Logistic Regression

Logistic Regression is a classification algorithm commonly used for problems where the output belongs to different classes.

In this project, the two classes are:

- 0 → Not Dark Pattern
- 1 → Dark Pattern

The model receives the TF-IDF numerical representation of the text and learns which features are associated with each class.

2. Multinomial Naive Bayes

Naive Bayes is a simple probabilistic classification algorithm that works well with text data.

It was selected because:

- It is simple to understand.
- It is commonly used for text classification.
- It works with numerical text features such as TF-IDF.
- It provides a useful comparison with Logistic Regression.

Model Comparison:

Model	Accuracy
Logistic Regression	92.80%
Naive Bayes	91.53%

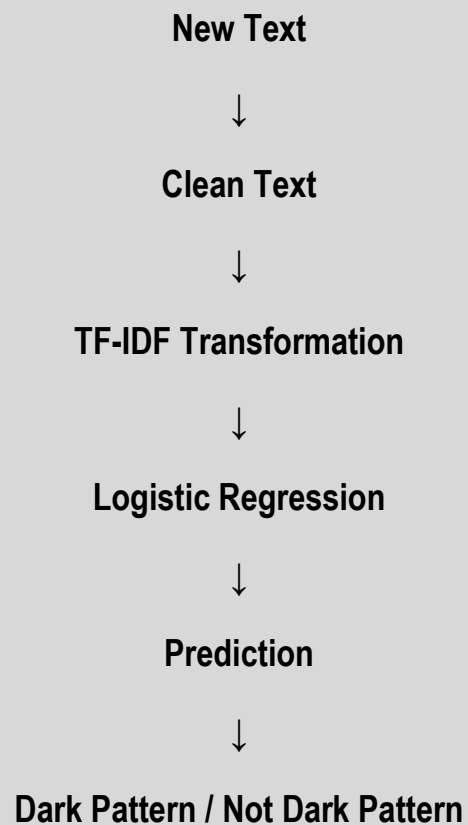
Logistic Regression performed slightly better in terms of overall accuracy.

Therefore:

Logistic Regression was selected as the final model.

- **Prediction:**

The prediction process is:



- **Evaluation:**

The final model was evaluated using:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

Implementation:

Programming Language:

The project was built using Python, in Google Colab.

Libraries:

We used Pandas to load and explore the dataset, re (regular expressions) for cleaning the text, Scikit-learn for splitting the data, building the models, and evaluating them, Matplotlib for plotting the confusion matrix, and Joblib to save the trained model so it doesn't need to be retrained every time.

NLP Techniques:

The raw text was cleaned first — converted to lowercase, stripped of special characters, and extra spaces removed. Then TF-IDF was used to turn this cleaned text into numbers the model can actually learn from, since machine learning models can't work with plain sentences.

Algorithm/Model:

Two models were trained and compared: Logistic Regression and Multinomial Naive Bayes. Logistic Regression gave a slightly higher accuracy (92.80% vs 91.53%), so it was chosen as the final model, even though Naive Bayes was a bit better at catching dark-pattern examples specifically.

Important Implementation Details:

The data was split 80% for training and 20% for testing, keeping the same balance of both classes on each side. A simple function called `detect_dark_pattern()` was written to tie the whole process together — clean the input text, convert it with TF-IDF, and return a prediction. Looking at which words most influenced the model's decisions (like "left," "only," "limited," "hurry"), it became clear the model was picking up on common promotional words rather than truly understanding the sentence — a useful reminder of the limits of this simple approach. Finally, the trained model and vectorizer were saved to disk so they could be reused without training again.

Results:

The developed system was evaluated using the test dataset. Two machine learning models were compared: Logistic Regression and Naive Bayes.

- **Model Accuracy:**

Model	Accuracy
Logistic Regression	92.80%
Naive Bayes	91.53%

...	Model	Accuracy	Accuracy (%)
0	Logistic Regression	0.927966	92.796610
1	Naive Bayes	0.915254	91.525424

- **Evaluation Metrics:**

Class	Precision	Recall	F1-Score
Not Dark Pattern	0.90	0.96	0.93
Dark Pattern	0.95	0.90	0.93

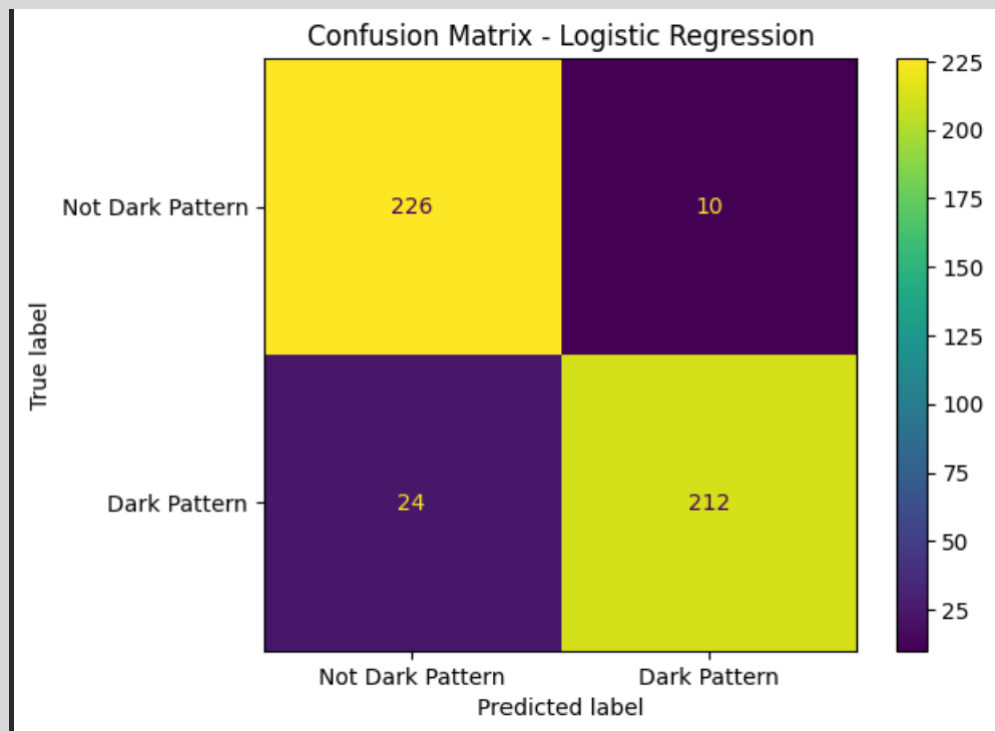
The model achieved good precision, recall, and F1-score for both classes.

- **Confusion Matrix:**

	Predicted Not Dark Pattern	Predicted Dark Pattern
Actual Not Dark Pattern	226	10
Actual Dark Pattern	24	212

The model correctly classified most of the test samples, with 438 out of 472 samples classified correctly

Figure 1: Confusion Matrix of Logistic Regression



- **Sample Outputs:**

```
Text: Hurry! Only 2 items left in stock!  
Prediction: Dark Pattern  
  
Text: Limited time offer. Buy now before the deal ends!  
Prediction: Dark Pattern  
  
Text: This product is available in three different sizes.  
Prediction: Dark Pattern  
  
Text: The product is made from high quality cotton.  
Prediction: Not Dark Pattern  
  
Text: Only 5 people are viewing this product right now!  
Prediction: Dark Pattern
```

The results show that the model can identify common dark-pattern language such as **urgency** and **scarcity**.

Limitations:

The project has some limitations:

- The dataset contains only **2,356 records**, so a larger dataset could improve the model.
- The model is mainly designed for **e-commerce text**.
- TF-IDF has limited understanding of the context and meaning of a complete sentence.

- Some unseen sentences may be incorrectly classified.
- The model cannot verify whether a statement such as *"Only 2 items left"* is actually true.
- The project focuses only on **text** and does not analyze webpage design, buttons, pop-ups, or visual elements.

Future Scope:

The project can be improved in the future by:

- Using a **larger and more diverse dataset**.
- Improving text preprocessing techniques.
- Using advanced NLP models such as **BERT or Transformer models**.
- Extending the system to identify specific dark-pattern categories such as **Scarcity, Urgency, and Social Proof**.
- Adding **multilingual support**.
- Developing a **web or mobile application**.
- Creating a **browser extension** for real-time detection.
- Combining text analysis with **webpage layout and visual information**.

Conclusion:

This project developed an NLP-based system for detecting dark-pattern language in e-commerce text. The text was cleaned using basic preprocessing techniques and converted into numerical features using TF-IDF.

Two machine learning models, Logistic Regression and Naive Bayes, were trained and compared. Logistic Regression achieved the best accuracy of 92.80%, while Naive Bayes achieved 91.53%.

Therefore, Logistic Regression was selected as the final model. The project shows that basic NLP and machine learning techniques can be effectively used to identify potentially manipulative language in e-commerce websites.

Through this project, we also learned that the model has limitations in understanding context, and further improvements can be made using larger datasets and advanced NLP techniques.

Submitted By:

Abhishek K B

24UBC203

3RD BCA B